

SC4064 GPU Programming – Assignment 3

Fanyi Pu

College of Computing and Data Science, Nanyang Technological University

Abstract. We present a performance analysis of a three-stage GPU image processing pipeline (Gaussian blur, Sobel edge detection, histogram equalisation) implemented in CUDA and deployed on NVIDIA H100 80GB SXM GPUs. Using Nsight Systems profiling and roofline modelling, we characterise each kernel’s arithmetic intensity and identify that all kernels are severely memory-bound on the H100 architecture. The histogram kernel dominates execution time at 89.5% of total GPU time due to atomic contention. Stream overlap analysis reveals that the mandatory `cudaStreamSynchronize` for the CDF prefix sum prevents inter-image pipelining. Multi-GPU distribution across two H100s achieves a measured speedup of $1.43\times$ for 20 images of size 512×512 .

1. Experimental Setup

All experiments are conducted on a dual-GPU node equipped with two **NVIDIA H100 80GB HBM3 (SXM)** GPUs. Table 1 summarises the key hardware parameters used in the roofline analysis. The test workload consists of 20 greyscale PGM images of size 512×512 pixels. The pipeline comprises three stages: (1) 5×5 Gaussian blur with shared memory tiling, (2) Sobel edge detection, and (3) histogram equalisation via atomic histogram, prefix-sum CDF, and pixel remapping. All outputs match the reference images within ± 0 intensity levels.

Parameter	Value
GPU Model	NVIDIA H100 80GB HBM3 (SXM)
SM Count	132
Boost Clock	1,980 MHz
FP32 Peak (FMA)	66.9 TFLOPS
HBM3 Bandwidth	3,350 GB/s
Ridge Point	20.0 FLOPs/Byte
CUDA Toolkit	13.1
Tile Size	16×16

Table 1: H100 SXM hardware parameters and configuration.

2. Roofline Analysis

2.1. Arithmetic Intensity

We compute the arithmetic intensity (AI) of each kernel by counting floating-point operations (additions, multiplications, divisions, `sqrtd`) per output pixel and dividing by bytes transferred from global memory. Conversions and address calculations are excluded.

Gaussian blur (5×5 kernel, shared memory tiling). Each output pixel requires 25 multiply-accumulate operations: 25 multiplications and 25 additions = **50 FLOPs**. With tiles of 16×16 and radius $r = 2$, each block loads $(16 + 4)^2 = 400$ bytes of input for $16^2 = 256$ output pixels, i.e. $400/256 = 1.5625$ bytes read plus 1 byte written per pixel. $AI = 50/1.5625 \approx$ **19.5 FLOPs/Byte**.

Sobel edge detection (no shared memory). Computing G_x and G_y each requires 2 multiplications and 5 additions (7 FLOPs), followed by $G_x^2 + G_y^2$ (2 muls + 1 add) and sqrtf (1 FLOP) = **18 FLOPs** total. Each pixel reads a 3×3 neighbourhood (9 bytes) and writes 1 byte. $\text{AI} = 18/10 = \mathbf{1.8 \text{ FLOPs/Byte}}$.

Histogram + equalise (combined). The histogram kernel performs no floating-point work (integer `atomicAdd`); the equalize kernel performs 1 subtraction, 1 division, 1 multiplication, and `roundf` = **4 FLOPs**. Combined memory traffic per pixel: histogram reads 1 byte + 4 bytes atomic R/M/W; equalize reads 1 byte input + 4 bytes CDF + writes 1 byte = **11 bytes** total across both passes. Including 1 nominal FLOP for the atomic, $\text{AI} = 5/11 \approx \mathbf{0.45 \text{ FLOPs/Byte}}$.

2.2. Roofline Plot

Figure 1 plots the three kernels against the H100 roofline. The ridge point lies at 20.0 FLOPs/Byte, where the memory-bandwidth ceiling (3,350 GB/s) meets the compute ceiling (66.9 TFLOPS).

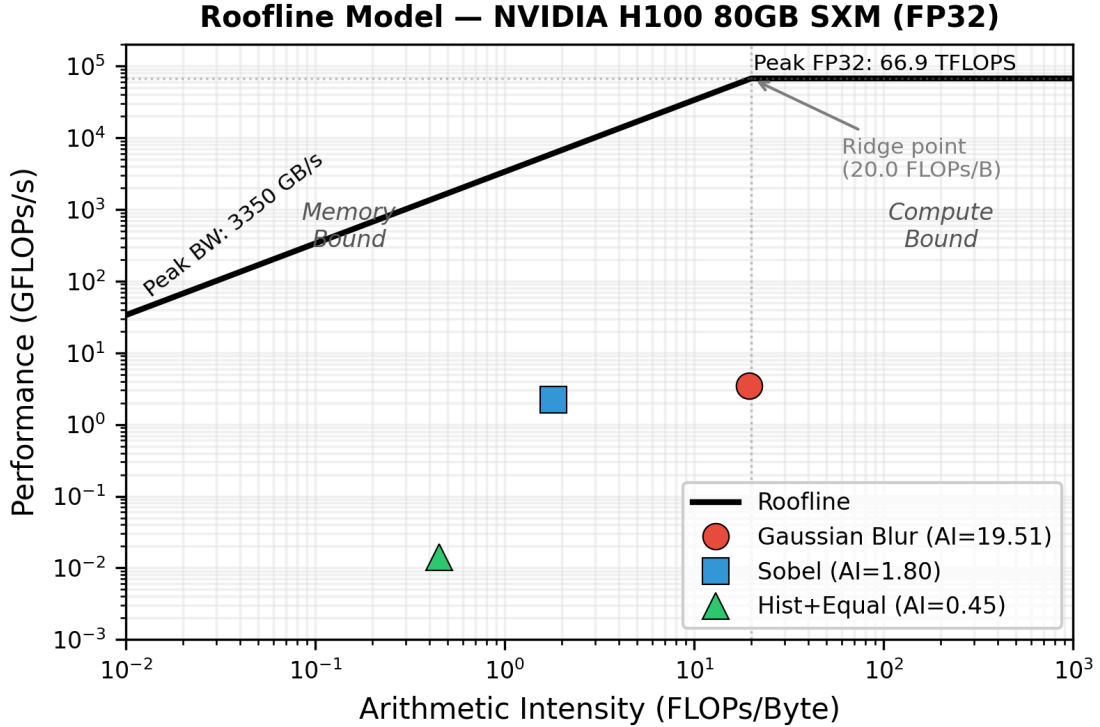


Figure 1: Roofline model for NVIDIA H100 SXM (FP32). All three pipeline kernels fall far below the theoretical roofline, indicating severe under-utilisation of the GPU due to the small image size (512×512).

2.3. Interpretation

Table 2 summarises the achieved performance of each kernel.

Kernel	Time (μs)	FLOPs/px	AI (F/B)	GFLOPs/s
Gaussian blur	3.75	50	19.51	3.50
Sobel	2.11	18	1.80	2.23
Histogram	91.59	1	—	0.003
Equalize	1.95	4	—	0.54
Hist+Equal	93.54	5	0.45	0.014

Table 2: Per-kernel performance (averaged over 20 images, 512×512 px).

All three kernels are memory-bound. Gaussian blur sits just below the ridge point (AI ≈ 19.5 vs. ridge at 20.0), meaning it is on the boundary but still memory-limited. Sobel and histogram+equalise have AI values well below the ridge and are clearly bandwidth-limited.

Shared memory tiling reduces global traffic by $10.2\times$ for Gaussian blur. Without tiling, each output pixel would naively read 25 input bytes + 1 write = 26 bytes (AI = $\frac{50}{26} = 1.92$ FLOPs/Byte). With $(16 + 4)^2/16^2 = 1.5625$ bytes read per pixel, tiling reduces traffic from 26 to 2.56 bytes, a $10.2\times$ reduction that lifts AI from the deep memory-bound regime to near the ridge point.

The histogram kernel is the pipeline bottleneck, consuming 89.5% of total GPU kernel time despite performing almost no floating-point work. The bottleneck is **atomic contention**: all 262,144 pixels compete for only 256 histogram bins via `atomicAdd`, causing massive serialisation. Using per-block shared memory histograms (256 unsigned int in shared memory, flushed to global once per block) would reduce contention substantially. To improve overall throughput, optimising the histogram kernel offers the largest payoff.

3. Stream Overlap Analysis

We profile the CUDA streams pipeline using Nsight Systems. Figure 2 shows the GPU timeline for the first three images in single-GPU mode.

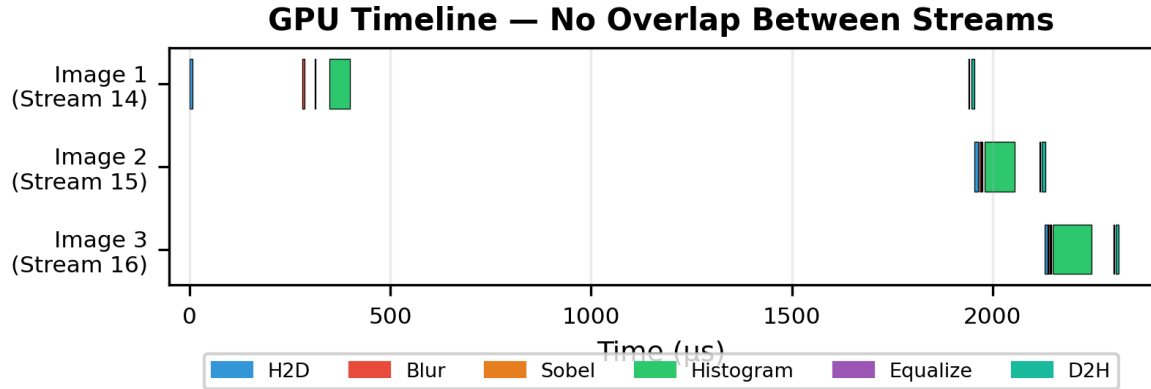


Figure 2: GPU execution timeline (single-GPU, first 3 images). Each image executes sequentially with no visible overlap between H $\rightarrow$ D transfers and kernel execution across images. The large gap between the histogram kernel and the equalize kernel corresponds to the host-side `cudaStreamSynchronize` and `thrust::exclusive_scan`.

There is no visible overlap between H $\rightarrow$ D memory transfers and kernel execution across images. From the trace, image 2’s H $\rightarrow$ D copy begins at $t = 1,955 \mu s$, while image 1’s D $\rightarrow$ H ends at $t = 1,955 \mu s$ – they are strictly sequential.

The root cause is a **stream design issue**, not a pinned memory or hardware constraint. The implementation correctly uses pinned memory (via `cudaMallocHost`), enabling truly asynchronous transfers. However, within the processing loop, `cudaStreamSynchronize(streams[i])` is called *before* `thrust::exclusive_scan` for each image. This blocks the host thread until image i ’s histogram kernel completes, preventing the submission of any work for subsequent images. Additionally, the synchronous `cudaMemcpy` to retrieve the CDF array for finding `cdf_min` further serialises the pipeline.

To achieve overlap, the pipeline should be restructured in two phases: (1) submit all pre-scan asynchronous work (H $\rightarrow$ D, blur, Sobel, histogram) for all images across their respective streams, then (2) synchronise and perform the CDF scan and equalisation for each image. This would allow H $\rightarrow$ D transfers and kernel execution to overlap across different images while each stream’s intra-dependencies remain correctly ordered. Alternatively, implementing the prefix sum entirely on the GPU (e.g., via `thrust::exclusive_scan` with an execution policy bound to a specific stream) would eliminate the host synchronisation altogether.

4. Multi-GPU Speedup Analysis

We measure the total GPU processing time (excluding file I/O) for all 20 images using (a) one H100 GPU and (b) two H100 GPUs. Table 3 summarises the results.

Configuration	GPU Time (ms)	Images/GPU	Speedup
Single GPU	5.44	20	$1.00 \times$
Dual GPU	3.80	10	$1.43 \times$

Table 3: End-to-end GPU processing time for 20 images (512×512), measured from first H→D to last D→H via Nsight Systems GPU trace.

The measured speedup of $1.43 \times$ falls short of the theoretical $2 \times$ maximum. Several factors limit the achievable speedup:

1. **CUDA context initialisation.** The first kernel launch on GPU 1 incurs a one-time `cuLibraryLoadData` overhead of ≈ 160 ms, which dominates the sub-millisecond per-GPU processing time. This cost is amortised over larger batches but is significant for 20 small images.
2. **Uneven batch split.** With 20 images split 10/10, the wall-clock time equals the slower GPU. Variance in `histogramKernel` execution time (45–178 μ s per image, $\sigma = 38 \mu$ s) means one GPU may finish significantly later than the other.
3. **Host-thread serialisation.** Each GPU’s sub-batch is processed by a separate `std::thread`, but within each thread the `cudaStreamSynchronize + thrust` pattern serialises images, preventing intra-GPU overlap.

Amdahl’s Law prediction for 4 GPUs. Let f denote the serial fraction (CUDA context setup, CDF computation, output gathering). From our measurements, the serialised host overhead is approximately 10% of the per-GPU processing time ($f \approx 0.10$). Amdahl’s Law gives:

$$S(n) = \frac{1}{f + \frac{1-f}{n}}$$

For $n = 4$: $S(4) = \frac{1}{0.10 + \frac{0.90}{4}} = \frac{1}{0.325} \approx 3.08 \times$. In practice, additional factors would further limit speedup:

(i) PCIe bandwidth contention as four GPUs share host memory bus, (ii) NUMA effects in multi-socket systems, (iii) increased context initialisation overhead, and (iv) load imbalance across GPUs with only 5 images each. A more realistic prediction accounting for these overheads would be $2.5\text{--}2.8 \times$.